មេរៀនទី ១១.២

អំពី មុខងារ Object (Object Method)

មុខងារ Object គឺជាគោលគំនិតដ៏សំខាន់មួយនៅក្នុងភាសា JavaScript ដែលអនុញ្ញាតឱ្យអ្នករៀបចំទិន្នន័យ និងសកម្មភាពនៅក្នុងទម្រង់ដែលមានរចនាសម្ព័ន្ធច្បាស់លាស់។ នៅពេលអ្នកចង់រក្សាទុកព័ត៌មានផ្សេងៗដែលទាក់ទងគ្នា ដូចជាឈ្មោះ អាយុ និងអាសយដ្ឋានរបស់មនុស្សម្នាក់ ឬព័ត៌មានអំពីផលិតផលមួយ ការប្រើប្រាស់ Object គឺជាវិធីដ៏សមស្របបំផុត។ Object មួយអាចមានលក្ខណៈសម្បត្តិ (Properties) ដែលជាទិន្នន័យ និងមុខងារ (Methods) ដែលជាសកម្មភាពដែល Object នោះអាចអនុវត្តបាន។ ការយល់ដឹងពីរបៀបបង្កើត និងប្រើប្រាស់ Object នឹងជួយឱ្យអ្នកសរសេរកូដដែលមានរចនាសម្ព័ន្ធល្អ និងងាយស្រួលថែរក្សា។

Made with GAMMA

មេរៀនទី ១១.៣

អំពី this Keyword

នៅក្នុង JavaScript this គឺជា Keyword ពិសេសដែលសំដៅលើ Object ដែលកំពុងប្រតិបត្តិមុខងារនៅពេលនោះ។ វាជាគោលគំនិតដ៏សំខាន់ដែលអ្នកគ្រប់គ្នាដែលរៀន JavaScript ត្រូវតែយល់ច្បាស់ ព្រោះវាបង្ហាញពី "អ្នកណា" កំពុងហៅមុខងារនោះ។ តម្លៃនៃ this អាចប្រែប្រួលអាស្រ័យលើរបៀបដែលមុខងារត្រូវបានហៅ។ ឧទាហរណ៍ នៅពេលអ្នកហៅមុខងារមួយដែលស្ថិតនៅក្នុង Object មួយ this នឹងសំដៅលើ Object នោះ។ ប៉ុន្តែប្រសិនបើអ្នកហៅមុខងារដោយឯករាជ្យ this នឹងសំដៅលើ Object ផ្សេងទៀត ដូចជា window នៅក្នុង Browser។

និយមន័យសាមញ្ញ

this សំដៅលើ Object ដែល "ធ្វើ" ឬ "ហៅ" មុខងារ។ វាដូចជាពាក្យថា "ខ្ញុំ" នៅពេលអ្នកនិយាយ — វាសំដៅលើអ្នកដែលកំពុងនិយាយ។

- នៅក្នុង Object Method → សំដៅលើ Object នោះ
- នៅក្នុង Function ធម្មតា → សំដៅលើ Window/Global
- នៅក្នុង Arrow Function → ទទួលមរតកពី Context ឪពុកម្តាយ
- នៅក្នុង Event Handler → សំដៅលើ Element ដែលបង្ក Event

ឧទាហរណ៍ក្នុងការអនុវត្ត

```
const person = {
  name: "សុខា",
  age: 25,
  greet: function() {
    console.log(
      "ខ្ញុំឈ្មោះ " + this.name
    );
  }
};

person.greet();
// លទ្ធផល: ខ្ញុំឈ្មោះ សុខា
// this សំដៅលើ person object
```

នៅក្នុងឧទាហរណ៍ខាងលើ អ្នកនឹងឃើញថា this.name សំដៅលើលក្ខណៈសម្បត្តិ name នៃ Object person។ នេះគឺជាអ្វីដែលធ្វើឱ្យ this មានប្រយោជន៍ខ្លាំង — វាអនុញ្ញាតឱ្យមុខងារចូលប្រើប្រាស់ទិន្នន័យរបស់ Object ដែលវាស្ថិតនៅ។ ប្រសិនបើអ្នកមិនប្រើ this ទេ អ្នកនឹងត្រូវចងចាំឈ្មោះ Object នៅពេលសរសេរមុខងារនីមួយៗ ដែលធ្វើឱ្យកូដមានភាពរឹងមាំតិចតួច។

ក្រៅពីនេះ this ក៏មានឥរិយាបថខុសៗគ្នាទៅតាមបរិបទផ្សេងៗទៀត។ ឧទាហរណ៍ នៅពេលអ្នកប្រើ setTimeout ឬ setInterval តម្លៃនៃ this អាចប្រែប្រួល ព្រោះមុខងារត្រូវបានហៅក្នុងបរិបទថ្មីមួយ។ នៅក្នុង Arrow Functions this មិនមានតម្លៃផ្ទាល់ខ្លួនទេ វាទទួលមរតកតម្លៃពីបរិបទដែលវាត្រូវបានបង្កើត។ ការយល់ដឹងពីភាពខុសគ្នាទាំងនេះគឺជាគន្លឹះដើម្បីចៀសវាងកំហុសដែលកើតឡើងញឹកញាប់នៅក្នុងការសរសេរកូដ JavaScript។

នៅពេលអ្នកធ្វើការជាមួយ Event Listeners នៅក្នុង DOM this នឹងសំដៅលើ HTML Element ដែលបង្ក Event នោះ។ ឧទាហរណ៍ ប្រសិនបើអ្នកចុចលើប៊ូតុងមួយ this នៅក្នុង Event Handler នឹងសំដៅលើប៊ូតុងនោះ។ នេះធ្វើឱ្យអ្នកអាចចូលប្រើប្រាស់លក្ខណៈសម្បត្តិផ្សេងៗរបស់ Element នោះបានយ៉ាងងាយស្រួល។

🎯 ក្នុង Object Method

this = Object ដែលមានមុខងារនោះ។ ឧទាហរណ៍ person.greet() → this = person

🌐 ក្នុង Global Scope

this = window (នៅក្នុង Browser) ឬ global (នៅក្នុង Node.js)

➡️ ក្នុង Arrow Function

this ទទួលមរតកពី Context ឪពុកម្តាយ — មិនមានតម្លៃផ្ទាល់ខ្លួនទេ

🖱️ ក្នុង Event Handler

this = HTML Element ដែលបង្ក Event នោះ (ឧទាហរណ៍ ប៊ូតុងដែលត្រូវបានចុច)

Made with GAMMA

មេរៀនទី ១១.៣ — បន្ត

ការគ្រប់គ្រង this ជាមួយ bind, call និង apply

JavaScript ផ្តល់នូវវិធីសាស្ត្របីយ៉ាងសម្រាប់គ្រប់គ្រងតម្លៃនៃ this ដោយចេតនា។ វិធីសាស្ត្រទាំងនេះគឺ bind(), call() និង apply()។ ពួកវាអនុញ្ញាតឱ្យអ្នកកំណត់ថាតើ this គួរតែសំដៅលើ Object ណាមួយ ទោះបីជាមុខងារនោះមិនមែនជាផ្នែកមួយរបស់ Object នោះក៏ដោយ។ ការយល់ដឹងពីភាពខុសគ្នារវាងវិធីសាស្ត្រទាំងបីនេះគឺជាជំនាញដ៏សំខាន់សម្រាប់អ្នកអភិវឌ្ឍន៍ JavaScript គ្រប់រូប។

bind()

បង្កើតមុខងារថ្មីដែលមាន this ត្រូវបានកំណត់ជាមុន។ មុខងារនេះមិនត្រូវបានហៅភ្លាមៗទេ ប៉ុន្តែត្រូវបានត្រឡប់មកវិញសម្រាប់ប្រើប្រាស់នៅពេលក្រោយ។ វាមានប្រយោជន៍នៅពេលអ្នកត្រូវការបញ្ជូនមុខងារទៅកាន់កន្លែងផ្សេង ដោយចង់រក្សាតម្លៃ this ដើម។

call()

ហៅមុខងារភ្លាមៗ ដោយកំណត់ this និងបញ្ជូន Arguments ម្តងមួយៗ។ វាមានប្រយោជន៍នៅពេលអ្នកចង់ហៅមុខងារមួយដោយផ្ទាល់ ហើយចង់កំណត់ថាតើ this គួរសំដៅលើ Object ណា។

apply()

ស្រដៀងនឹង call() ប៉ុន្តែទទួល Arguments ជា Array។ វាមានប្រយោជន៍នៅពេលអ្នកមានទិន្នន័យជាទម្រង់ Array ហើយចង់បញ្ជូនវាទៅកាន់មុខងារមួយ។

ដើម្បីយល់កាន់តែច្បាស់ សូមពិនិត្យមើលឧទាហរណ៍ខាងក្រោម។ នៅពេលអ្នកមានមុខងារមួយដែលត្រូវការប្រើប្រាស់នៅក្នុងបរិបទផ្សេងៗគ្នា bind() អាចជួយរក្សាតម្លៃ this ដែលអ្នកត្រូវការ។ ឧទាហរណ៍ ប្រសិនបើអ្នកចង់ប្រើមុខងារមួយជា Event Handler ប៉ុន្តែចង់ឱ្យ this នៅតែសំដៅលើ Object ដើម អ្នកអាចប្រើ bind() ដើម្បីចង this ទៅនឹង Object នោះជាមុនសិន។

```
const person = {
  name: "វិចិត្រ",
  greet: function(greeting) {
    console.log(greeting + ", ខ្ញុំឈ្មោះ " + this.name);
  }
};

const greetFunc = person.greet.bind(person);
greetFunc("សួស្តី"); // សួស្តី, ខ្ញុំឈ្មោះ វិចិត្រ

// call() - ហៅភ្លាមៗ
person.greet.call(person, "ជំរាបសួរ");

// apply() - ហៅភ្លាមៗ ដោយបញ្ជូន args ជា array
person.greet.apply(person, ["សួស្តី"]);
```

ភាពខុសគ្នាសំខាន់រវាង call() និង apply() គឺត្រង់របៀបដែលពួកវាទទួល Arguments។ call() ទទួល Arguments ម្តងមួយៗ ដូចជា func.call(obj, arg1, arg2) ចំណែកឯ apply() ទទួល Arguments ជាទម្រង់ Array ដូចជា func.apply(obj, [arg1, arg2])។ ការជ្រើសរើសរវាងពីរនេះអាស្រ័យលើទម្រង់ទិន្នន័យដែលអ្នកមាន។

មូលហេតុដែល bind() មានសារៈសំខាន់ខ្លាំងនៅក្នុងការអភិវឌ្ឍន៍គេហទំព័រគឺដោយសារតែវាជួយដោះស្រាយបញ្ហា this ដែលបាត់បង់នៅពេលប្រើមុខងារជា Callback។ ឧទាហរណ៍ នៅពេលអ្នកប្រើ setTimeout ឬបន្ថែម Event Listener មុខងារដែលអ្នកបញ្ជូនទៅនោះអាចបាត់បង់តម្លៃ this ដើមរបស់វា។ ការប្រើ bind() ធានាថា this នៅតែសំដៅលើ Object ដែលអ្នកចង់បាន។

> 💡 **គន្លឹះសំខាន់៖** ប្រសិនបើអ្នកមានបញ្ហាជាមួយ this ក្នុង Callback ឬ Event Handler សូមព្យាយាមប្រើ bind() ឬប្រើ Arrow Function ដែលទទួលមរតក this ដោយស្វ័យប្រវត្តិ។

Made with GAMMA

មេរៀនទី ១១.៤

អំពី Array Object

Array គឺជា Object ពិសេសមួយនៅក្នុង JavaScript ដែលត្រូវបានប្រើសម្រាប់រក្សាទុកបញ្ជីនៃតម្លៃ។ វាមានលក្ខណៈពិសេសដោយសារតែវាអាចផ្ទុកទិន្នន័យច្រើននៅក្នុងអថេរតែមួយ ហើយមានមុខងារជាច្រើនដែលអាចអនុវត្តលើវាបាន។ Array គឺជាឧបករណ៍មួយដែលអ្នកនឹងប្រើប្រាស់ជារៀងរាល់ថ្ងៃនៅពេលសរសេរកូដ JavaScript។ ឧទាហរណ៍ ប្រសិនបើអ្នកចង់រក្សាទុកឈ្មោះសិស្សទាំងអស់ក្នុងថ្នាក់ ឬបញ្ជីផលិតផលនៅក្នុងហាងអនឡាញ Array គឺជាជម្រើសដ៏សមស្របបំផុត។

របៀបបង្កើត Array

```
const fruits = ["ផ្លែប៉ោម", "ចេក", "ស្វាយ"];
const numbers = [1, 2, 3, 4, 5];
const mixed = ["ខ្មែរ", 2024, true];

// ចូលប្រើ Element
console.log(fruits[0]); // ផ្លែប៉ោម
console.log(fruits.length); // 3
```

លក្ខណៈពិសេសនៃ Array

- អាចផ្ទុកទិន្នន័យគ្រប់ប្រភេទ
- មានលក្ខណៈសម្បត្តិ `length`
- មានមុខងារជាច្រើនសម្រាប់គ្រប់គ្រង
- អាចផ្លាស់ប្តូរទំហំបាន (Mutable)
- Index ចាប់ផ្តើមពី 0

មុខងារសំខាន់ៗនៃ Array

Array នៅក្នុង JavaScript មានមុខងារជាច្រើនដែលជួយអ្នកគ្រប់គ្រង និងធ្វើការជាមួយទិន្នន័យបានយ៉ាងមានប្រសិទ្ធភាព។ មុខងារនីមួយៗមានតួនាទីខុសៗគ្នា ហើយការយល់ដឹងពីពួកវានឹងជួយឱ្យអ្នកសរសេរកូដបានកាន់តែខ្លី និងមានប្រសិទ្ធភាព។ ខាងក្រោមនេះគឺជាមុខងារសំខាន់ៗដែលអ្នកគួរស្គាល់៖

push() / pop()

`push()` បន្ថែម Element នៅចុង Array ចំណែក `pop()` យក Element ចុងចេញ។ ឧទាហរណ៍ `fruits.push("ទុរេន")` នឹងបន្ថែម "ទុរេន" នៅចុងបញ្ជី។

filter()

ច្រោះយកតែ Element ដែលត្រូវនឹងលក្ខខណ្ឌ។ ឧទាហរណ៍ អ្នកអាចច្រោះយកតែលេខដែលធំជាង ១០ ពី Array មួយ។

map()

បំប្លែងគ្រប់ Element ទៅជាតម្លៃថ្មី។ ឧទាហរណ៍ អ្នកអាចគុណលេខទាំងអស់ក្នុង Array នឹង ២ ដើម្បីបង្កើត Array ថ្មីមួយ។

find() / findIndex()

ស្វែងរក Element ដំបូងដែលត្រូវនឹងលក្ខខណ្ឌ។ `find()` ត្រឡប់តម្លៃ ចំណែក `findIndex()` ត្រឡប់ Index របស់វា។

sort() / reverse()

`sort()` តម្រៀប Element តាមលំដាប់ ចំណែក `reverse()` ត្រឡប់លំដាប់នៃ Array។ អ្នកអាចបញ្ជាក់ Function សម្រាប់កំណត់របៀបតម្រៀប។

reduce()

បូកសរុប ឬបង្រួម Array ទៅជាតម្លៃតែមួយ។ ឧទាហរណ៍ អ្នកអាចប្រើវាដើម្បីគណនាផលបូកនៃលេខទាំងអស់ក្នុង Array។

មុខងារទាំងនេះគឺជាឧបករណ៍ដ៏មានឥទ្ធិពលដែលអ្នកនឹងប្រើប្រាស់ញឹកញាប់នៅក្នុងការអភិវឌ្ឍន៍គេហទំព័រ។ ជាពិសេស `map()`, `filter()` និង `reduce()` ត្រូវបានគេប្រើយ៉ាងទូលំទូលាយនៅពេលធ្វើការជាមួយទិន្នន័យ។ ឧទាហរណ៍ ប្រសិនបើអ្នកកំពុងបង្កើតគេហទំព័រដែលបង្ហាញបញ្ជីផលិតផល អ្នកអាចប្រើ `map()` ដើម្បីបង្កើត HTML Element សម្រាប់ផលិតផលនីមួយៗ។

```
const prices = [10, 20, 30, 40];

// map() - បំប្លែងគ្រប់តម្លៃ
const doubled = prices.map(p => p * 2);
// [20, 40, 60, 80]

// filter() - ច្រោះតម្លៃ
const expensive = prices.filter(p => p > 25);
// [30, 40]

// reduce() - បូកសរុប
const total = prices.reduce((sum, p) => sum + p, 0);
// 100
```

Made with GAMMA

មេរៀនទី ១២

អំពី DOM Manipulation

តើ DOM ជាអ្វី?

DOM ឬ **Document Object Model** គឺជាតំណាងនៃទំព័រ HTML របស់អ្នកក្នុងទម្រង់ជា Object ដែល Browser បង្កើតឡើង។ នៅពេល Browser អាន និងបង្ហាញទំព័រ HTML វាបង្កើតរចនាសម្ព័ន្ធមួយដែលហៅថា DOM Tree ដែលមាន Node ជាច្រើនតំណាងឱ្យ Element នីមួយៗនៅក្នុងទំព័រ។ រចនាសម្ព័ន្ធនេះមានលក្ខណៈដូចជាមែកឈើ ដែលមានឫស (Root) នៅខាងលើ ហើយមានមែក និងស្លឹករឹកចេញពីនោះ។ DOM គឺជាចំណុចប្រទាក់រវាង JavaScript និងទំព័រ HTML របស់អ្នក — វាអនុញ្ញាតឱ្យ JavaScript អាន ផ្លាស់ប្តូរ បន្ថែម ឬលុប Element នៅក្នុងទំព័របាន។

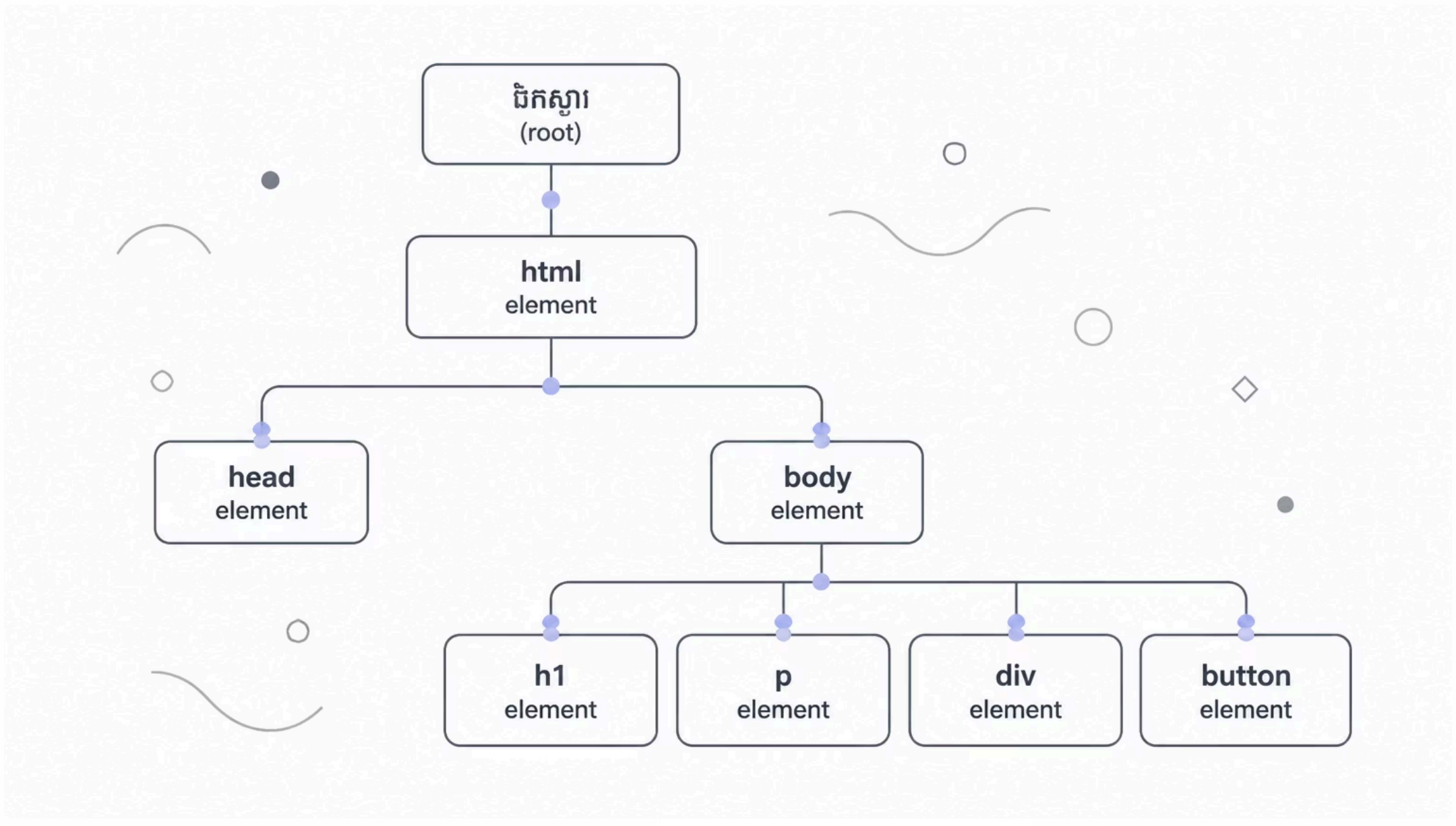
សម្រាប់អ្នកចាប់ផ្តើមរៀន គិតថា DOM ដូចជា "ផែនទី" របស់ទំព័រ HTML របស់អ្នក។ រាល់ tag នីមួយៗនៅក្នុង HTML ដូចជា `<h1>`, `<p>`, `<div>` ត្រូវបានបំប្លែងទៅជា Object មួយនៅក្នុង DOM។ JavaScript អាចចូលប្រើប្រាស់ Object ទាំងនេះ ហើយធ្វើការផ្លាស់ប្តូរលើពួកវាបាន។ នេះហើយជាមូលហេតុដែលអ្នកអាចប្រើ JavaScript ដើម្បីផ្លាស់ប្តូរអត្ថបទ ពណ៌ ទំហំ ឬសូម្បីតែបន្ថែម Element ថ្មីៗទៅក្នុងទំព័របាន ដោយមិនចាំបាច់ Reload ទំព័រឡើងវិញ។

📄 HTML Source	🌳 DOM Tree	🖥️ Rendered Page
កូដ HTML ដើមដែលអ្នកសរសេរនៅក្នុង File	រចនាសម្ព័ន្ធដើមឈើដែល Browser បង្កើតឡើង	ទំព័រដែលអ្នកប្រើឃើញនៅលើអេក្រង់

DOM មានរចនាសម្ព័ន្ធជាប្រភេទដើមឈើ (Tree Structure) ដែលមាន `document` ជាឫស (Root Node)។ ពីនោះ វាមាន Node កូនៗជាច្រើន រួមមាន `<html>`, `<head>`, `<body>` និង Element ផ្សេងៗទៀត។ រាល់ Element នីមួយៗគឺជា Node មួយ ហើយអាចមាន Node កូនៗរបស់វាផ្ទាល់។ ការយល់ដឹងពីរចនាសម្ព័ន្ធនេះគឺជាគន្លឹះដើម្បីធ្វើការជាមួយ DOM បានយ៉ាងមានប្រសិទ្ធភាព។

នៅពេលអ្នកធ្វើការជាមួយ DOM អ្នកនឹងជួបប្រទះនឹងគំនិតសំខាន់ៗមួយចំនួន។ **Node** គឺជាពាក្យទូទៅសម្រាប់គ្រប់ផ្នែកទាំងអស់នៅក្នុង DOM Tree — រួមទាំង Element, Attribute, និង Text។ **Element** គឺជា Node ប្រភេទមួយដែលតំណាងឱ្យ HTML Tag។ **Parent** គឺជា Node ដែលមាន Node កូននៅខាងក្រោម ចំណែក **Child** គឺជា Node ដែលស្ថិតនៅខាងក្រោម Parent។ **Sibling** គឺជា Node ដែលមាន Parent តែមួយ។ ការយល់ដឹងពីទំនាក់ទំនងទាំងនេះនឹងជួយឱ្យអ្នកធ្វើដំណើរក្នុង DOM Tree បានយ៉ាងងាយស្រួល។

មូលហេតុដែល DOM Manipulation មានសារៈសំខាន់ខ្លាំងនៅក្នុងការអភិវឌ្ឍន៍គេហទំព័រទំនើប គឺដោយសារតែវាអនុញ្ញាតឱ្យអ្នកបង្កើតទំព័រដែលមានភាពរស់រវើក និងឆ្លើយតបទៅនឹងសកម្មភាពរបស់អ្នកប្រើប្រាស់។ ឧទាហរណ៍ នៅពេលអ្នកចុចលើប៊ូតុង "Like" នៅលើ Facebook ទំព័រមិនចាំបាច់ Reload ទេ — JavaScript ប្រើ DOM Manipulation ដើម្បីផ្លាស់ប្តូរចំនួន Like ភ្លាមៗ។ នេះគឺជាអ្វីដែលធ្វើឱ្យគេហទំព័រទំនើបមានអារម្មណ៍រហ័ស និងរលូន។



រូបភាពខាងលើបង្ហាញពីរចនាសម្ព័ន្ធដើមឈើនៃ DOM។ `document` ស្ថិតនៅខាងលើបំផុតជាឫស បន្ទាប់មក `html` ជា Node កូន ហើយ `head` និង `body` ជា Node កូនរបស់ `html`។ Element ទាំងអស់ដែលអ្នកឃើញនៅលើទំព័រ សុទ្ធតែស្ថិតនៅក្នុង `body`។ ការយល់ដឹងពីរចនាសម្ព័ន្ធនេះនឹងជួយឱ្យអ្នកដឹងពីរបៀបធ្វើដំណើរ និងចូលប្រើប្រាស់ Element ផ្សេងៗនៅក្នុងទំព័ររបស់អ្នក។

Made with GAMMA

មេរៀនទី ១២.១

ការជ្រើសរើស Element

មុននឹងអ្នកអាចធ្វើការផ្លាស់ប្តូរអ្វីមួយនៅលើទំព័រ អ្នកត្រូវតែ "ជ្រើសរើស" Element ដែលអ្នកចង់ធ្វើការជាមួយសិន។ JavaScript ផ្តល់នូវវិធីសាស្ត្រជាច្រើនសម្រាប់ជ្រើសរើស Element ហើយវិធីសាស្ត្រនីមួយៗមានគុណសម្បត្តិ និងការប្រើប្រាស់ខុសៗគ្នា។ ការស្គាល់វិធីសាស្ត្រទាំងនេះនឹងជួយឱ្យអ្នកជ្រើសរើសវិធីសាស្ត្រដែលសមស្របបំផុតសម្រាប់ស្ថានភាពនីមួយៗ។

getElementById()

ជ្រើសរើស Element តាម ID តែមួយ។ វាជាវិធីសាស្ត្រចាស់ ប៉ុន្តែនៅតែមានប្រសិទ្ធភាពខ្ពស់។

```
const title = document.getElementById("main-title");
title.style.color = "blue";
```

- ត្រឡប់ Element តែមួយ
- លឿន និងសាមញ្ញ
- គ្រប់គ្រងដោយ ID តែមួយ

querySelector() / querySelectorAll()

ជ្រើសរើស Element ដោយប្រើ CSS Selector។ វាជាវិធីសាស្ត្រទំនើប និងមានភាពបត់បែនខ្ពស់។

```
const firstItem = document.querySelector(".item");
const allItems = document.querySelectorAll(".item");
```

- ប្រើ CSS Selector បានគ្រប់ប្រភេទ
- `querySelector()` ត្រឡប់ Element ដំបូង
- `querySelectorAll()` ត្រឡប់ NodeList

ភាពខុសគ្នាសំខាន់រវាង `getElementById()` និង `querySelector()` គឺត្រង់ភាពបត់បែន។ `getElementById()` អាចជ្រើសរើសបានតែតាម ID ប៉ុណ្ណោះ ប៉ុន្តែវាលឿនជាងបន្តិច។ `querySelector()` អាចជ្រើសរើសបានតាម Class, Tag Name, Attribute ឬ Selector ស្មុគស្មាញផ្សេងៗទៀត។ ឧទាហរណ៍ អ្នកអាចប្រើ `querySelector("div.container > p:first-child")` ដើម្បីជ្រើសរើស Element តាមលក្ខខណ្ឌស្មុគស្មាញ។

ក្រៅពីវិធីសាស្ត្រទាំងពីរនេះ ក៏មានវិធីសាស្ត្រផ្សេងៗទៀតដែលអ្នកគួរស្គាល់។ `getElementsByClassName()` ជ្រើសរើស Element ទាំងអស់ដែលមាន Class ជាក់លាក់មួយ។ `getElementsByTagName()` ជ្រើសរើស Element ទាំងអស់ដែលមាន Tag Name ជាក់លាក់។ វិធីសាស្ត្រទាំងនេះត្រឡប់ HTMLCollection ដែលមានលក្ខណៈ "Live" — មានន័យថាប្រសិនបើ DOM ផ្លាស់ប្តូរ HTMLCollection នឹងអាប់ដេតដោយស្វ័យប្រវត្តិ។

ប្រៀបធៀបវិធីសាស្ត្រជ្រើសរើស

`getElementById()` → លឿន សាមញ្ញ ប៉ុន្តែជ្រើសរើសបានតែតាម ID

`querySelector()` → បត់បែន ទំនើប ប្រើ CSS Selector បាន

`querySelectorAll()` → ជ្រើសរើស Element ច្រើនក្នុងពេលតែមួយ

`getElementsByClassName()` → ជ្រើសរើសតាម Class (ត្រឡប់ Live Collection)

គន្លឹះអនុវត្ត

ប្រើ `querySelector()` សម្រាប់ភាគច្រើននៃករណី ព្រោះវាមានភាពបត់បែនខ្ពស់។ ប្រើ `getElementById()` នៅពេលអ្នកចង់បានល្បឿន និងជ្រើសរើសតាម ID តែមួយ។ ចៀសវាងការប្រើ `document.all` ព្រោះវាចាស់ហើយមិនត្រូវបានណែនាំ។

នៅពេលអ្នកជ្រើសរើស Element រួចហើយ អ្នកអាចរក្សាទុកវានៅក្នុងអថេរមួយ ហើយប្រើប្រាស់វាដើម្បីធ្វើការផ្លាស់ប្តូរផ្សេងៗ។ វាជាការអនុវត្តដ៏ល្អក្នុងការរក្សាទុក Element ដែលអ្នកជ្រើសរើសញឹកញាប់នៅក្នុងអថេរ ដើម្បីចៀសវាងការហៅ `querySelector()` ច្រើនដងពេក ដែលអាចធ្វើឱ្យកូដរបស់អ្នកមានប្រសិទ្ធភាពថយចុះ។

Made with GAMMA

មេរៀនទី ១២.២

ការផ្លាស់ប្តូរ Content, Style និង Attribute

បន្ទាប់ពីអ្នកបានជ្រើសរើស Element រួចហើយ ជំហានបន្ទាប់គឺការធ្វើការផ្លាស់ប្តូរលើវា។ JavaScript អនុញ្ញាតឱ្យអ្នកផ្លាស់ប្តូរអត្ថបទ ពណ៌ ទំហំ និង Attribute ផ្សេងៗ របស់ Element បានយ៉ាងងាយស្រួល។ នេះគឺជាមូលដ្ឋានគ្រឹះនៃការបង្កើតគេហទំព័រដែលមានភាពរស់រវើក និងមានអន្តរកម្មជាមួយអ្នកប្រើប្រាស់។

ផ្លាស់ប្តូរ Content

`textContent` ផ្លាស់ប្តូរអត្ថបទសុទ្ធ

`innerHTML` ផ្លាស់ប្តូរ HTML ខាងក្នុង

```
el.textContent = "សួស្តី!";
el.innerHTML = "<b>សួស្តី!</b>";
```

ផ្លាស់ប្តូរ Style

ប្រើ `style` property

CSS property ប្រើ camelCase

```
el.style.color = "red";
el.style.fontSize = "20px";
el.style.backgroundColor =
"#f0f0f0";
```

ផ្លាស់ប្តូរ Attribute

`setAttribute()` កំណត់ Attribute

`getAttribute()` អាន Attribute

```
el.setAttribute("src",
"image.jpg");
const src =
el.getAttribute("src");
```

ភាពខុសគ្នារវាង `textContent` និង `innerHTML` គឺសំខាន់ណាស់។ `textContent` ផ្លាស់ប្តូរតែអត្ថបទសុទ្ធ ហើយវាមានសុវត្ថិភាពជាងព្រោះវាមិនអនុញ្ញាតឱ្យមាន HTML Tag នៅក្នុងអត្ថបទទេ។ `innerHTML` អនុញ្ញាតឱ្យអ្នកបញ្ចូល HTML Tag ប៉ុន្តែវាអាចបង្កគ្រោះថ្នាក់ប្រសិនបើអ្នកប្រើវាជាមួយទិន្នន័យដែលមកពីអ្នកប្រើប្រាស់ (User Input) ព្រោះវាអាចនាំឱ្យមានការវាយប្រហារ XSS (Cross-Site Scripting)។ ដូច្នេះ សូមប្រើ `textContent` នៅពេលដែលអ្នកគ្រាន់តែចង់ផ្លាស់ប្តូរអត្ថបទ។

នៅពេលផ្លាស់ប្តូរ Style តាមរយៈ JavaScript អ្នកត្រូវប្រើ camelCase សម្រាប់ CSS Property។ ឧទាហរណ៍ `background-color` ក្លាយជា `backgroundColor`, `font-size` ក្លាយជា `fontSize`, និង `border-radius` ក្លាយជា `borderRadius`។ នេះគឺដោយសារតែ JavaScript មិនអនុញ្ញាតឱ្យប្រើសញ្ញាដក (-) នៅក្នុងឈ្មោះ Property។ ប្រសិនបើអ្នកចង់ផ្លាស់ប្តូរ Class របស់ Element អ្នកអាចប្រើ `classList.add()`, `classList.remove()` ឬ `classList.toggle()` ដែលជាវិធីសាស្ត្រដ៏ស្អាត និងងាយស្រួល។

ឧទាហរណ៍ពេញលេញ

```
const box = document.querySelector(".box");

// ផ្លាស់ប្តូរអត្ថបទ
box.textContent = "ខ្ញុំត្រូវបានផ្លាស់ប្តូរ!";

// ផ្លាស់ប្តូរ Style
box.style.color = "white";
box.style.backgroundColor = "#373B48";
box.style.padding = "20px";
box.style.borderRadius = "8px";

// បន្ថែម/ដក Class
box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("highlight");

// ផ្លាស់ប្តូរ Attribute
const img = document.querySelector("img");
img.setAttribute("alt", "រូបភាពថ្មី");
img.setAttribute("width", "300");
```

គន្លឹះសំខាន់ៗ

- ប្រើ `textContent` ជំនួសឱ្យ `innerHTML` នៅពេលមិនត្រូវការ HTML
- ប្រើ `classList` សម្រាប់គ្រប់គ្រង Class ជំនួសឱ្យការកំណត់ `className` ដោយផ្ទាល់
- CSS Property ត្រូវសរសេរជា camelCase នៅក្នុង JavaScript
- ប្រើ `style.cssText` សម្រាប់កំណត់ Style ច្រើនក្នុងពេលតែមួយ

Made with GAMMA

មេរៀនទី ១២.៣

ការបង្កើត និងលុប Element ថ្មី

JavaScript មិនត្រឹមតែអាចផ្លាស់ប្តូរ Element ដែលមានស្រាប់ប៉ុណ្ណោះទេ វាក៏អាចបង្កើត Element ថ្មីៗ និងលុប Element ចាស់ៗចេញពីទំព័របានផងដែរ។ នេះគឺជាសមត្ថភាពដ៏មានឥទ្ធិពលដែលអនុញ្ញាតឱ្យអ្នកបង្កើតទំព័រដែលអាចផ្លាស់ប្តូរទៅតាមសកម្មភាពរបស់អ្នកប្រើប្រាស់ ដោយមិនចាំបាច់ Reload ទំព័រឡើងវិញ។ ឧទាហរណ៍ នៅពេលអ្នកបន្ថែម Comment ថ្មីនៅលើ Facebook ឬបង្ហាញលទ្ធផលស្វែងរកថ្មីៗ នោះគឺជាលទ្ធផលនៃការបង្កើត Element ថ្មីតាមរយៈ JavaScript។



ដំណើរការនៃការបង្កើត Element ថ្មីមាន ៤ ជំហានសំខាន់ៗ។ ទីមួយ អ្នកត្រូវបង្កើត Element ដោយប្រើ `createElement()`។ ទីពីរ អ្នកកំណត់អត្ថបទ ឬ HTML ខាងក្នុងរបស់វា។ ទីបី អ្នកជ្រើសរើស Parent Element ដែលអ្នកចង់បន្ថែមវាចូល។ ទីបួន អ្នកបន្ថែមវាទៅក្នុងទំព័រដោយប្រើ `appendChild()` ឬ `insertBefore()`។ ដំណើរការនេះគឺសាមញ្ញ ប៉ុន្តែមានសារៈសំខាន់ខ្លាំងនៅក្នុងការអភិវឌ្ឍន៍គេហទំព័រ។

បង្កើត Element ថ្មី

```
// បង្កើត Element ថ្មី
const newDiv = document.createElement("div");

// កំណត់ Content
newDiv.textContent = "ខ្ញុំជា Div ថ្មី!";

// កំណត់ Class និង ID
newDiv.className = "box";
newDiv.id = "my-box";

// កំណត់ Style
newDiv.style.color = "blue";

// បន្ថែមទៅក្នុង Page
const container = document.querySelector(".container");
container.appendChild(newDiv);
```

លុប Element

វិធីទី ១ — ប្រើ Parent:

```
const item = document.querySelector(".item");
item.parentNode.removeChild(item);
```

វិធីទី ២ — ប្រើ remove() (ទំនើប):

```
const item = document.querySelector(".item");
item.remove();
```

លុប Element ទាំងអស់ក្នុង Container:

```
container.innerHTML = "";
```

វិធីសាស្ត្រ `appendChild()` បន្ថែម Element ថ្មីនៅចុងបញ្ចប់នៃ Parent Element។ ប្រសិនបើអ្នកចង់បន្ថែម Element នៅទីតាំងជាក់លាក់មួយ អ្នកអាចប្រើ `insertBefore()`។ ឧទាហរណ៍ `parent.insertBefore(newElement, referenceElement)` នឹងបន្ថែម `newElement` នៅពីមុខ `referenceElement`។ វិធីសាស្ត្រ `replaceChild()` អនុញ្ញាតឱ្យអ្នកជំនួស Element មួយដោយ Element មួយទៀត។

នៅពេលលុប Element វិធីសាស្ត្រ `remove()` គឺជាវិធីសាស្ត្រទំនើប និងងាយស្រួលប្រើបំផុត។ វាអនុញ្ញាតឱ្យអ្នកលុប Element ដោយផ្ទាល់ ដោយមិនចាំបាច់សំដៅលើ Parent របស់វា។ ប៉ុន្តែប្រសិនបើអ្នកត្រូវការគាំទ្រ Browser ចាស់ៗ អ្នកអាចប្រើវិធីសាស្ត្រចាស់គឺ `parentNode.removeChild()`។ ការប្រើ `innerHTML = ""` គឺជាវិធីដ៏រហ័សក្នុងការលុប Element ទាំងអស់នៅក្នុង Container មួយ ប៉ុន្តែវាមិនល្អសម្រាប់ប្រសិទ្ធភាពទេ ព្រោះវាត្រូវបង្កើត និងកម្ទេច Node ទាំងអស់ក្នុងពេលតែមួយ។

> ⚠️ **ចំណាំ៖** នៅពេលលុប Element ដែលមាន Event Listener ភ្ជាប់ជាមួយ សូមប្រាកដថាអ្នកបានដក Event Listener ចេញជាមុនសិន (ប្រសិនបើចាំបាច់) ដើម្បីចៀសវាង Memory Leak។

Made with GAMMA

មេរៀនទី ១២.៤

អំពី Event Listeners

Event Listener គឺជាមុខងារដែល "ស្តាប់" សកម្មភាពរបស់អ្នកប្រើប្រាស់ ហើយអនុវត្តកូដនៅពេលដែល Event មួយកើតឡើង។ Event អាចជាអ្វីក៏បាន ដូចជាការចុច Mouse, ការវាយ Keyboard, ការផ្លាស់ប្តូរទំហំ Window, ឬការបញ្ចូលទិន្នន័យក្នុង Form។ ដោយគ្មាន Event Listener ទំព័ររបស់អ្នកនឹងគ្រាន់តែជាទំព័រស្ងៀមដែលមិនអាចឆ្លើយតបទៅនឹងសកម្មភាពរបស់អ្នកប្រើប្រាស់បានទេ។ Event Listener គឺជាបេះដូងនៃភាពរស់រវើកនៅក្នុងគេហទំព័រទំនើប។

click

កើតឡើងនៅពេលអ្នកប្រើចុចលើ Element។ ជា Event ដែលប្រើញឹកញាប់បំផុត សម្រាប់ប៊ូតុង និង Link។

keydown / keyup

`keydown` កើតឡើងនៅពេលចុច Key ចុះ ចំណែក `keyup` កើតឡើងនៅពេលលែង Key។

submit / change

`submit` កើតឡើងនៅពេល Submit Form ចំណែក `change` កើតឡើងនៅពេលតម្លៃនៃ Input ផ្លាស់ប្តូរ។

mouseover / mouseout

`mouseover` កើតឡើងនៅពេល Mouse ចូលលើ Element ចំណែក `mouseout` កើតឡើងនៅពេល Mouse ចេញ។

របៀបបន្ថែម Event Listener គឺដោយប្រើមុខងារ `addEventListener()`។ មុខងារនេះទទួលប៉ារ៉ាម៉ែត្រពីរ ឬបី៖ ឈ្មោះនៃ Event (ដូចជា "click"), Function ដែលនឹងត្រូវហៅនៅពេល Event កើតឡើង (ហៅថា Callback Function), និងប៉ារ៉ាម៉ែត្របន្ថែមមួយទៀតដែលជាជម្រើស (Optional) សម្រាប់កំណត់លក្ខណៈពិសេស។ ការប្រើ `addEventListener()` គឺជាវិធីសាស្ត្រទំនើប និងត្រូវបានណែនាំជាងការប្រើ `onclick` ឬ Attribute ផ្សេងៗនៅក្នុង HTML។

```
const button = document.querySelector("#myButton");

// បន្ថែម Event Listener
button.addEventListener("click", function() {
  console.log("ប៊ូតុងត្រូវបានចុច!");
});

// ប្រើ Arrow Function
button.addEventListener("click", () => {
  console.log("ប៊ូតុងត្រូវបានចុចម្តងទៀត!");
});

// Event ជាមួយ Parameter (Event Object)
button.addEventListener("click", (event) => {
  console.log(event.target); // Element ដែលបង្ក Event
  console.log(event.type);   // ឈ្មោះ Event
});
```

នៅក្នុងឧទាហរណ៍ខាងលើ អ្នកនឹងឃើញថា Callback Function ទទួលបាន **Event Object** ជាប៉ារ៉ាម៉ែត្រ។ Event Object នេះមានព័ត៌មានជាច្រើនអំពី Event ដែលកើតឡើង ដូចជា `event.target` (Element ដែលបង្ក Event), `event.type` (ឈ្មោះនៃ Event), `event.clientX` និង `event.clientY` (ទីតាំងនៃ Mouse)។ ការយល់ដឹងពី Event Object នឹងជួយឱ្យអ្នកអាចគ្រប់គ្រង Event បានកាន់តែច្បាស់លាស់។

មុខងារ `preventDefault()` គឺជាមុខងារមួយដ៏សំខាន់ដែលអនុញ្ញាតឱ្យអ្នកបញ្ឈប់សកម្មភាពធម្មតានៃ Event មួយ។ ឧទាហរណ៍ នៅពេលអ្នក Submit Form ទំព័រនឹង Reload ដោយស្វ័យប្រវត្តិ។ ប្រសិនបើអ្នកចង់បញ្ឈប់សកម្មភាពនេះ អ្នកអាចប្រើ `event.preventDefault()`។ ដូចគ្នាដែរ សម្រាប់ Link នៅពេលអ្នកចុចលើវា ទំព័រនឹងផ្លាស់ប្តូរទៅ URL ថ្មី — ប្រសិនបើអ្នកចង់បញ្ឈប់សកម្មភាពនេះ អ្នកក៏អាចប្រើ `preventDefault()` បានដែរ។

preventDefault() និង stopPropagation()

`event.preventDefault()` — បញ្ឈប់សកម្មភាពធម្មតានៃ Event

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
  // ទំព័រមិន Reload
});
```

`event.stopPropagation()` — បញ្ឈប់ការរីករាលដាលនៃ Event ទៅ Parent

ដក Event Listener

នៅពេលអ្នកមិនត្រូវការ Event Listener ទៀតទេ អ្នកគួរតែដកវាចេញដើម្បីសន្សំសំចៃ Memory។

```
function handleClick() {
  console.log("ចុចហើយ!");
}

// បន្ថែម
button.addEventListener("click", handleClick);

// ដកចេញ
button.removeEventListener("click", handleClick);
```

> ⚠️ ត្រូវប្រាកដថា Function ដែលប្រើក្នុង `removeEventListener` គឺដូចគ្នាទៅនឹង Function ដែលប្រើក្នុង `addEventListener`។ Arrow Function មិនអាចដកចេញបានទេ ព្រោះវាជា Function ថ្មីរាល់ដង។

ការយល់ដឹងពី Event Listeners គឺជាជំនាញមូលដ្ឋានដ៏សំខាន់សម្រាប់អ្នកអភិវឌ្ឍន៍គេហទំព័រ។ វាអនុញ្ញាតឱ្យអ្នកបង្កើតទំព័រដែលឆ្លើយតបទៅនឹងសកម្មភាពរបស់អ្នកប្រើប្រាស់ ធ្វើឱ្យបទពិសោធន៍របស់ពួកគេកាន់តែប្រសើរឡើង។ ចាប់ពីប៊ូតុងដែលផ្លាស់ប្តូរពណ៌នៅពេលចុច រហូតដល់ Form ដែលផ្ទៀងផ្ទាត់ទិន្នន័យមុននឹង Submit — ទាំងអស់នេះគឺជាលទ្ធផលនៃ Event Listeners។ សូមអនុវត្តជំនាញទាំងនេះឱ្យបានញឹកញាប់ ហើយអ្នកនឹងក្លាយជាអ្នកអភិវឌ្ឍន៍ JavaScript ដ៏ពូកែម្នាក់!

Made with GAMMA

មេរៀនទី ១១.២

អំពី មុខងារ Object (Object Method)

មុខងារ Object គឺជាគោលគំនិតដ៏សំខាន់មួយនៅក្នុងភាសា JavaScript ដែលអនុញ្ញាតឱ្យអ្នករៀបចំទិន្នន័យ និងសកម្មភាពនៅក្នុងទម្រង់ដែលមានរចនាសម្ព័ន្ធច្បាស់លាស់។ នៅពេលអ្នកចង់រក្សាទុកព័ត៌មានផ្សេងៗដែលទាក់ទងគ្នា ដូចជាឈ្មោះ អាយុ និងអាសយដ្ឋានរបស់មនុស្សម្នាក់ ឬព័ត៌មានអំពីផលិតផលមួយ ការប្រើប្រាស់ Object គឺជាវិធីដ៏សមស្របបំផុត។ Object មួយអាចមានលក្ខណៈសម្បត្តិ (Properties) ដែលជាទិន្នន័យ និងមុខងារ (Methods) ដែលជាសកម្មភាពដែល Object នោះអាចអនុវត្តបាន។ ការយល់ដឹងពីរបៀបបង្កើត និងប្រើប្រាស់ Object នឹងជួយឱ្អកសរសេរកូដដែលមានរចនាសម្ព័ន្ធល្អ និងងាយស្រួលថែរក្សា។

Made with GAMMA